

UNIVERSIDAD AUTÓNOMA DE CAMPECHE
FACULTAD DE INGENIERÍA

PRÁCTICA 1

RACE CONDITION

Materia - Concurrencia y Paralelismo
Ingeniería en Tecnologías de Software

Alumno: Diego Valadez Almeyda
Docente: Sergio A. Noh Puch
Fecha: 26/09/2026

DESARROLLO



CONDICION DE CARRERA

Una condición de carrera ocurre cuando dos o más hilos acceden y modifican un mismo dato compartido sin una sincronización adecuada. El resultado final puede depender del orden en que los hilos realizan sus operaciones.

¿POR QUÉ PUEDE PRODUCIR RESULTADOS DIFERENTES AL EJECUTAR VARIAS VECES?

Porque el sistema operativo es quien decide cuándo cada hilo obtiene tiempo de ejecución.

Por ejemplo, la secuencia podría ejecutarse como:

Hilo 1, Hilo 1, Hilo 2, Hilo 2.

Mientras que en otro caso podría ser:

Hilo 1, Hilo 2, Hilo 1, Hilo 2.

SECCION CRITICA

Es el fragmento de código donde múltiples hilos o procesos intentan acceder, leer y modificar un mismo recurso simultáneamente. En el código anexo, la sección crítica se encuentra de forma iterativa dentro de los ciclos de ejecución de ambas rutinas de los hilos (`thread_routine` y `thread_routine_two`).

DESARROLLO



CÓDIGO ENSAMBLADOR

```
.L7:
    leaq    mutex(%rip), %rax
    movq    %rax, %rcx
    call    pthread_mutex_lock
    movl    global_counter(%rip), %eax
    subl    $1, %eax
    movl    %eax, global_counter(%rip)
    movl    global_counter(%rip), %edx
    leaq    .LC4(%rip), %rcx
    leaq    -112(%rbp), %rax
    movl    %edx, %r9d
    movq    %rcx, %r8
    movl    $100, %edx
    movq    %rax, %rcx
    call    __mingw_snprintf
    leaq    .LC2(%rip), %rax
    movl    $420, %r8d
    movl    $265, %edx
    movq    %rax, %rcx
    call    open
    movl    %eax, -12(%rbp)
    leaq    -112(%rbp), %rax
    movq    %rax, %rcx
    call    strlen
    movl    %eax, %ecx
    leaq    -112(%rbp), %rdx
    movl    -12(%rbp), %eax
    movl    %ecx, %r8d
    movl    %eax, %ecx
    call    write
    movl    -12(%rbp), %eax
    movl    %eax, %ecx
    call    close
    leaq    mutex(%rip), %rax
    movq    %rax, %rcx
    call    pthread_mutex_unlock
    addl    $1, -4(%rbp)
.L6:
```

DESARROLLO



QUÉ SIGNIFICA MOVL, ADDL Y SUBL?

En el código se pueden ver las siguientes instrucciones:

```
1. movl    global_counter(%rip), %eax
2. subl    $1, %eax
3. movl    %eax, global_counter(%rip)
4. addl    $1, -4(%rbp)
```

movl es utilizado para mover o copiar datos, donde el orden está definido por el registro **%eax**, entonces en el 1er **movl** se toma el valor de **global_counter** y se copia en el registro **%eax**. En el segundo **movl** pasa lo contrario.

subl y **addl** son operaciones fundamentales de las matemáticas donde **subl** realiza una resta y **addl** realiza una suma. La combinación de estas instrucciones es lo que permite incrementar y disminuir el contador global.

CONCLUSIONES

La practica permitió observar que una condición de carrera ocurre cuando varios hilos tratan de modificar una misma variable al mismo tiempo. El uso de mutex ayuda a evitar este problema protegiendo la sección crítica. Además, el análisis del código ensamblador permitió describir el cómo se modifica la variable **global_counter** mediante el uso de instrucciones como **movl**, **addl** y **subl**.

Diego Valadez